

第 17 章 能力、表示与上下文学习——LLM 为什么会“像是在推理”？

本章目标

“模型涌现出能力”“表示里出现概念”“上下文里学会了新任务”“长 CoT 就是推理”都是极容易被过度解释的句子。本章把这些叙述拆成可测量对象：**能力指标**、**隐藏表示**、**in-context posterior**、**算法模拟与推理轨迹**。我们将看到，ICL 可以被理解为对潜在任务的统计推断，也可以在特定设置下近似学习算法；所谓 emergence 可能来自真正的相变，也可能只是评价函数的阈值效应。

Prerequisite Recall | 先把这些知识唤醒

- 熟悉 Bayes posterior、线性回归和 Attention。
- 会区分模型参数更新与仅改变输入 context。
- 记住 probe 能解码信息不等于模型因果使用它。

Misconception Check | 本章最容易掉进去的坑

看到 benchmark 曲线突然跃升，就直接称为“涌现能力”，可能只是指标阈值造成的视觉跳变。先检查连续 latent performance、评分阈值、数据污染与任务进入训练分布等混杂。

Concept Contrast | 把相邻概念主动分开

	Decodability	Causal use
问题	hidden state 里能读出吗？	模型实际依赖它吗？
证据	probe / linear classifier	ablation / patching / intervention
逻辑关系	可解码是弱证据，不自动推出被用于计算	

Checkpoint | 开始正文前的 30 秒自测

ICL 时参数 θ 没变，为什么预测规则仍能变？**检查答案：**context 改变 forward computation 的中间状态，相当于在激活空间实现数据依赖算法，而非持久参数更新。

17.1 第一原则: Capability 是“系统在分布上的可测性能”

设任务分布为 $\mathcal{T}$ ，提示为 x ，系统（模型 + decoder + tools）为 S ，效用为 $u(S, x)$ 。一个最小定义是

$$\mathcal{C}(S; \mathcal{T}) = \mathbb{E}_{x \sim \mathcal{T}}[u(S, x)].$$

这个定义故意把 prompt、decoder 与 test-time compute 包含在系统 S 中。因为现实中的“模型能力”常常混合了模型权重、采样策略、context、检索和 verifier。

如果想研究权重本身，必须固定其余因素；如果想研究产品系统，就不能把系统提升全部归因于 base model。

17.2 “涌现”：连续潜变量也能产生跳变指标

假设某个潜在能力随 scale s 平滑改善：

$$q(s) = \sigma(a(s - s_0)).$$

若 benchmark 给分规则是“准确率超过阈值 τ 才算成功”，报告指标

$$m(s) = \mathbf{1}\{q(s) > \tau\}$$

会呈现突然从 0 到 1 的“涌现”。这并不证明底层机制存在相变。

更一般地，exact match、pass/fail compilation、majority vote 都可能把连续分布变化非线性地映射成能力曲线。因此研究 emergence 至少要同时观察：loss、概率、calibration、pass@ k 、连续 partial credit 与数据暴露程度。

Research Frontier | Training on the Test Task Confounds Evaluation and Emergence

会议身份：ICLR 2025 Oral。论文展示 task-relevant training 会混淆模型比较和 emergent capability 判断，并观察某些 emergence 随 test-task training 增加而变得平滑。

本章意义：所谓“能力突然出现”既可能是模型内部变化，也可能是数据轴或指标轴隐藏在图外。

论文入口：[ICLR Proceedings](#)

17.3 In-Context Learning：参数不变，预测规则却改变

给 demonstrations

$$D_m = \{(x_1, y_1), \dots, (x_m, y_m)\}$$

和 query x_* , ICL 的现象是模型在不更新权重的情况下, 输出

$$p_\theta(y_*|D_m, x_*)$$

随 demonstrations 改变。

这并不神秘: 任何条件概率模型在输入增加后都应改变后验预测。真正有研究价值的问题是: Transformer 学到了什么样的 **in-context algorithm**?

17.4 Bayesian 视角: 上下文是在推断潜在任务

设每个 episode 先采潜在任务 $z \sim p(z)$, 再由

$$(x_i, y_i) \sim p(x, y|z)$$

生成 demonstrations。理想 Bayesian learner 计算

$$p(z|D_m) \propto p(z) \prod_{i=1}^m p(y_i|x_i, z)$$

并预测

$$p(y_*|x_*, D_m) = \int p(y_*|x_*, z)p(z|D_m) dz.$$

于是 ICL 不需要“把知识写入参数”: 参数存的是**如何从 context 识别任务并预测**的元算法。

二元线性任务的手算

任务只有 $z \in \{+1, -1\}$, 数据满足

$$y = zx + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2).$$

均匀先验下, log posterior odds 是

$$\log \frac{p(z = +1|D)}{p(z = -1|D)} = \frac{1}{2\sigma^2} \sum_i [(y_i + x_i)^2 - (y_i - x_i)^2] = \frac{2}{\sigma^2} \sum_i x_i y_i.$$

因此 demonstrations 的充分统计量就是 $\sum_i x_i y_i$ 。一个 Transformer 若学会在 context 中聚合这种统计量, 就能表现出“读几个例子就理解规则”。

17.5 线性回归 ICL: posterior mean 是一个显式学习算法

设

$$y = Xw + \epsilon, \quad w \sim \mathcal{N}(0, \lambda^{-1}I), \quad \epsilon \sim \mathcal{N}(0, \sigma^2I).$$

Bayesian posterior mean 为

$$\mu_w = (X^\top X + \lambda \sigma^2 I)^{-1} X^\top y.$$

对 query x_* , 预测均值

$$\hat{y}_* = x_*^\top \mu_w.$$

这就是 ridge regression。于是一个 sequence model 若在 forward pass 中近似计算矩阵统计量、逆算子或其迭代近似，就可以在参数不更新的情况下模拟“临时训练一个回归器”。

17.6 Gradient-descent 视角：Attention 可以实现数据依赖更新

对平方损失

$$L(w) = \frac{1}{2} \sum_i (w^\top x_i - y_i)^2,$$

从 $w_0 = 0$ 做一步梯度下降：

$$w_1 = \eta \sum_i y_i x_i.$$

于是

$$\hat{y}_* = x_*^\top w_1 = \eta \sum_i y_i (x_*^\top x_i).$$

右侧正是“query 与 demonstration feature 的相似度加权求和”。这和 Attention 的计算骨架

$$\sum_i a(x_*, x_i) v_i$$

高度相似。需要谨慎：这并不证明所有真实 LLM 的 ICL 就是在“做梯度下降”，但它展示了 Transformer 为什么有能力表示这种学习算法。

Research Frontier | In-Context Learning Strategies Emerge Rationally

会议身份：NeurIPS 2025 Main Track。该工作从 rational analysis / hierarchical Bayesian 视角解释不同 ICL 策略为何会随训练条件出现：在任务混合下，memorizing predictor 与 generalizing predictor 可由一族 Bayesian predictors 统一刻画，并能很好预测 Transformer 训练中的 next-token 行为。

研究启示：与其问“ICL 是 Bayesian 还是 memorization”，不如问什么训练分布与计算约束使哪一种策略成为合理解？

官方入口：[NeurIPS 2025 program](#)

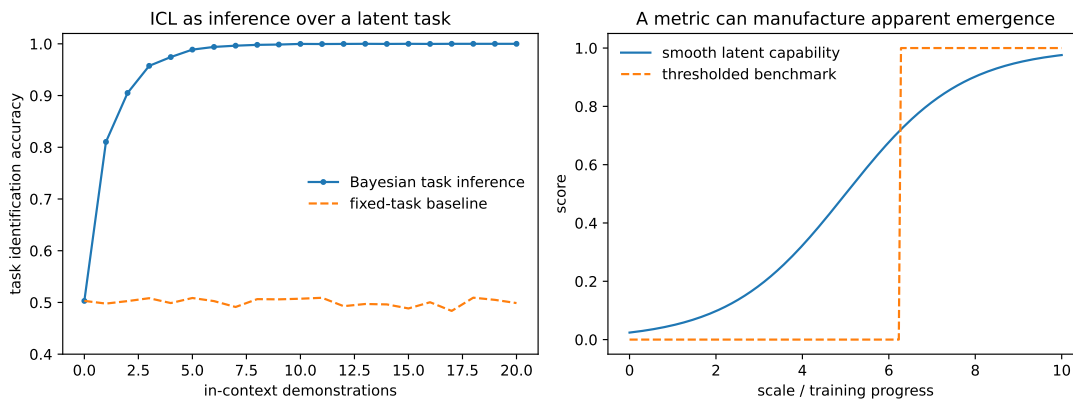


图 17.1 机制级实验：左图在二元潜在任务上展示 Bayesian ICL 如何随 demonstrations 累积快速识别 task；右图展示平滑潜在能力经过阈值指标后可以产生“突然涌现”的视觉假象。

17.7 表示是什么？Hidden state 不是“概念本身”

Transformer 第 l 层、第 t 个位置产生

$$h_t^{(l)} \in \mathbb{R}^d.$$

研究表示时常训练 probe

$$\hat{z} = \text{softmax}(Wh + b)$$

预测语言学属性、事实、计划或安全状态。如果 probe 准确，只能说明 z 可以从 h 中被某个函数读出；它不自动证明模型自身使用了这个信息。

要接近因果结论，通常需要 intervention：沿某个方向修改 h ，观察输出行为是否系统变化；或者做 activation patching、causal mediation、ablation。**decodability** 与 **causal use** 是两个命题。

17.8 低秩、子空间与 Superposition

输出 logits 为

$$z_t = W_U h_t + b,$$

所以对固定模型，若把多个 prompt 的 logits 堆成矩阵 Z ，它天然受到 hidden dimension 与表示几何限制。现实模型还可能表现出更强的 approximate low-rank structure。

与此同时，feature 数量可以远大于 residual dimension。Superposition 的思想是让多个稀疏 feature 共用维度，只要它们很少同时激活。于是表示研究的核心不应只是“第 123 维是什么”，而是**哪些方向/子空间/非线性结构承载可复用特征**。

Research Frontier | Sequences of Logits Reveal the Low Rank Structure of Language Models

会议身份：ICLR 2026 Oral。工作以模型无关的 sequential probabilistic model 视角，从不同 prompt/response 的 logits 矩阵研究 approximate low rank。

教材连接：第 2 章 SVD、第 7 章 logit geometry、第 17 章 representation 在这里汇合：低维结构不仅可能存在于 hidden state，也能在模型可观测输出函数层被检测。

官方入口：[ICLR 2026 Oral program](#)

17.9 Reasoning：把“答案正确”与“搜索过程”分开

对可验证任务， $\text{pass}@k$ 定义为 k 次采样中至少一个正确的概率。若 p 为独立单次成功率，理想化地

$$\text{pass}@k = 1 - (1 - p)^k.$$

$\text{pass}@1$ 提升可能来自分布 sharpen：正确轨迹原来就存在，但概率提高； $\text{pass}@k$ 在很大 k 下的上界则更接近“支持集中是否存在可解轨迹”。两者回答不同问题。

Chain-of-thought 还增加了中间 token，使模型获得额外 sequential compute 与可读 scratchpad。但“生成了步骤”不等于步骤对最终答案有因果贡献。Faithfulness、self-revision 和 hidden reasoning 都使解释更加困难。

17.10 长上下文 ICL 也存在统计效率问题

更多 demonstration 并非自动更好：context 变长后，模型可能出现位置偏置、检索干扰、示例冲突和 diminishing efficiency。

Research Frontier | Technical Debt in In-Context Learning: Diminishing Efficiency in Long Context

会议身份：NeurIPS 2025 Main Track。工作在 meta-ICL 回归设置中把 Transformer 与 Bayes-optimal learner 比较，发现 ICL 在短 context 可接近高效，但随着 context 变长，样本效率可能显著恶化，并给出信息论分析。

研究启示：context window 的物理长度不是有效学习样本数。

官方入口：[NeurIPS 2025 paper page](#)

Mini Lab | Frontier Lab 17: Bayesian ICL 与“假涌现”

运行 `labs/frontier/ch17_icl_bayes.py`。第一部分在 $y = \pm x + \epsilon$ 的 episode 分布上计算精确 posterior odds，展示 demonstration 如何让 task posterior 集中；第二部分把一个平滑 sigmoid latent capability 经过阈值指标，得到看似突然的

emergence。升级版：训练 tiny Transformer 做同一 meta-learning 任务，并把其预测与 Bayes posterior 逐步比较。

From Textbook to Research | 模型到底“学会了算法”，还是“压缩了任务分布”？

这两个说法并不互斥。一个足够好的概率模型可能通过压缩训练 episode 分布而表现出算法行为；研究关键是找到可区分的 intervention：改变 task prior、示例顺序、维度、噪声、context length 或 query distribution，观察模型是否遵循某个算法应有的系统性外推。

17.11 Deep Dive: Bayesian ICL 与 Algorithmic ICL 可以统一

Bayesian posterior 本身就是算法。区别只在我们从哪个层级描述：

概率层： $p(z|D)$ $\iff$ 算法层：怎样计算这个 posterior.

一个 Transformer 可以不显式存储“Bayes”符号，却通过 attention/MLP 学到近似 posterior update 的程序。因此“统计推断解释”和“算法模拟解释”不必互相排斥。

17.12 Deep Dive: 线性回归 posterior 的矩阵推导

先验与 likelihood 给出

$$-\log p(w|X, y) = \frac{1}{2\sigma^2} \|y - Xw\|^2 + \frac{\lambda}{2} \|w\|^2 + C.$$

展开二次项并配方，precision matrix 为

$$\Lambda = \frac{1}{\sigma^2} X^\top X + \lambda I,$$

posterior mean

$$\mu = \Lambda^{-1} \frac{1}{\sigma^2} X^\top y = (X^\top X + \lambda \sigma^2 I)^{-1} X^\top y.$$

这与 ridge estimator 完全一致。于是“看到几个例子后学习线性任务”可以有精确 statistical target，而不是只看 benchmark accuracy。

17.13 Deep Dive: 为什么 linear probe 容易被误读？

高维表示中，很多变量即使没有被模型主动使用，也可能与 hidden state 相关。一个参数量很大的 probe 甚至可以从噪声中拟合复杂边界。因此至少要控制：

1. probe capacity;
2. train/test split 是否跨分布;
3. selectivity/control task;
4. intervention 后行为是否改变。

Probe 是测量仪器，不是因果证明。

17.14 Deep Dive: 所谓“推理边界”依赖 sampling definition

若一个模型给正确轨迹极小但非零概率，理论上 $k \rightarrow \infty$ 时 $\text{pass}@k \rightarrow 1$ ；但现实预算有限。于是“能力边界”必须附带资源定义：最大 token、最大 sample、temperature、工具调用次数。没有资源约束的“模型能不能解”往往不是可操作问题。

17.14.1 Support、Coverage、Concentration

可以把 reasoning distribution 分成三个层面：

1. **Support**：好轨迹是否存在于可采样空间；
2. **Coverage**：好轨迹总概率质量有多少；
3. **Concentration**：概率是否集中到少数高质量轨迹。

预训练主要塑造前两者，后训练经常显著改变第三者；但现实论文正在争论 RLVR 是否也会扩展 support。这为下一章埋下直接问题。

研究者应问什么？

当论文声称“模型学会了 reasoning”，先问：pass@1 还是 pass@1024 提升？base distribution 中是否已经存在同类轨迹？表示 probe 是否有 causal intervention？benchmark 是否被训练任务覆盖？把这些问题答清楚，才可能区分 capability acquisition 与 probability reallocation。

17.15 Proof & Worked Example Clinic

Dependency Check | 先修与跨章连接

ICL 的 latent-state 视角与 第 20 章 POMDP belief 有直接数学同构，但前者的“task”通常是隐变量而非环境物理 state。

Worked Example | Bayesian ICL 的最小可计算例子

线性任务 $y = wx + \epsilon$, 先验 $w \sim \mathcal{N}(0, \tau^2)$ 、噪声 $\epsilon \sim \mathcal{N}(0, \sigma^2)$ 。观察设计向量 x 与标签 y 后, posterior variance

$$\Sigma_w = (\tau^{-2} + \sigma^{-2} x^T x)^{-1},$$

posterior mean

$$\mu_w = \Sigma_w \sigma^{-2} x^T y.$$

更多 in-context examples 本质上不断更新 latent task 的 posterior; Attention 是否真的实现这个算法, 则是经验与机制问题。

Proof Clinic | 从一步 GD 到 attention-like estimator

平方损失从 $w_0 = 0$ 做一步梯度下降:

$$w_1 = \eta \sum_i y_i x_i.$$

对 query x_* , 预测

$$\hat{y}_* = x_*^T w_1 = \eta \sum_i y_i (x_*^T x_i).$$

这已经具有“query 与 examples 做相似度, 再加权 value”的结构。它不是 Transformer=gradient descent 的证明, 却展示了为何 attention 足以表示某些学习算法。

Research Assignment | ICL 机制辨识

设计 latent-task mixture: 让 Bayesian posterior、nearest-neighbor 和 one-step GD 对同一 prompt 给出不同预测。训练小 Transformer 后用 intervention 区分更像哪类策略, 而不是只比较最终 accuracy。

最低交付: 1 页假设与指标声明; 可复现代码与固定 seed; 主结果图; 至少一个失败/反例; 200–400 字结论边界。**禁止:** 只报告最好一次 run、只给截图、把 mechanism-level reproduction 表述为 paper-scale replication。

研究阅读与复现检查表

1. capability 的单位是什么: accuracy、reward、pass@k、calibration 还是 sample efficiency?
2. system comparison 是否固定了 decoder、context 与 test-time compute?
3. emergence 是否在连续指标上仍存在?
4. ICL 理论对改变 task prior、noise、dimension 有什么可证伪预测?

5. probe 只证明 decodability, 还是有 intervention 支持 causal use?
6. reasoning gain 是 support expansion、coverage increase 还是 concentration/sharpening?
7. long-context ICL 的物理 context 长度与有效样本数是否被区分?

分层习题

分层训练 | Level A | 概念与计算

1. 说明“probe 能读出变量”为什么不等于“模型使用变量”。
2. 对二元任务例子推导 posterior odds, 并解释 $\sum_i x_i y_i$ 的作用。
3. 构造一个连续能力经过 exact-match 阈值后表现为突然跃迁的例子。
4. 解释 pass@1 与大 k pass@ k 分别更接近哪种能力概念。

分层训练 | Level B | 推导

1. 推导 Bayesian ridge posterior mean。
2. 从 $w_0 = 0$ 推导一次梯度下降后的预测, 并指出它与 attention weighted sum 的对应项。
3. 若 demonstrations 中存在两个 latent task family, 写出 posterior predictive 的 mixture 形式。
4. 证明 $Z = HW_U^\top$ 的 rank 不超过 $\min(\text{rank}(H), \text{rank}(W_U))$ 。

分层训练 | Level C | 研究设计

1. 设计一个 intervention 区分“模型记住 demonstration 格式”与“模型推断 latent task”。
2. 为某个 reasoning benchmark 设计连续评分, 减少 exact-match 阈值造成的假 emergence。
3. 设计 causal probe: 先发现表示方向, 再干预该方向验证行为变化。
4. 比较两种 ICL 理论时, 列出至少三个可以产生不同预测的实验条件。